Unit III :: Control Structures and Functions

SEC-01-PHY-91-R-03

Dr. Parag Bhattacharya
Department of Physics
Rangapara College (Autonomous)

Syllabus

Unit III: Control Structures and Functions

Conditional statements: 'if-else' statement; loops: 'for' loop, 'while' loop, breaking and continuing; functions: defining and calling functions in script files or inline: argument passing, return values; modular programming and code reuse: organizing code into reusable functions, documentation best practices; applications: writing programmes for series sums, iterative algorithms for numerical methods.

1 Conditional statements

Conditionals allow a program to choose between different actions by evaluating boolean expressions and executing only the block whose test is true. Using `if`, `elseif` and `else` constructs, a program can implement multiple branches so that different inputs or situations lead to different behaviour. They are widely used for input validation, error handling and implementing branching algorithms, making code more flexible and robust.

1.1 The `if-elseif-else` statement

The `if-else` construct executes a block of code when a condition is true and another block when it is false. Furthermore, the `elseif` construct provides additional branches. We use this whenever the program must choose between alternatives.

The general syntax of the `if-elseif-else` statement is as follows:

```
if condition then
// statements executed when condition is true
elseif other_condition then
// statements when other_condition is true
else
// statements when none of the above conditions hold
end
```

When using this conditional statement in Scilab, it is important to note the following:

- The keyword `then` is required after the condition.
- The `if-elseif-else` block must be closed by the keyword `end`.
- Conditions must evaluate to a scalar logical value.
- Use == for equality and ~= for inequality.
- Other comparison operators are >, <, >=, <=.
- Logical operators include & (AND) and | (OR).

Example: *to determine whether a given number is positive or negative or zero*

```
a = input("Enter a number: ");
if a > 0 then
disp("Positive");
elseif a < 0 then
disp("Negative");
else
disp("Zero");
end
```

2 Iterative loops

Scilab provides `for` loops for counted iteration, i.e. they run over index ranges or explicit vectors and are ideal when the number of repetitions is known in advance. On the other hand, `while` loops repeat as long as a condition holds and are suited to tasks that stop on runtime criteria (for example convergence tests). The control statements `break` and `continue` let the program alter normal loop flow.

2.1 The `for` loop

This iterative loop is used to repeat a block a known number of times or iterate over an index set or explicit vector.

The general syntax of the `for` loop is as follows:

```
for i = start:step:end
// body using i
end

// or iterate over a vector of values
for v = [1 3 5 7]
// body using v
end
```

When using this iterative loop in Scilab, it is important to note the following:

- The `step` keyword is optional; if only `start:end` is mentioned then `step` is taken to be 1 by default.
- It is possible to count down with a negative step, e.g. `10:-1:1`.
- It is preferable to preallocate arrays (e.g. `A=zeros(1,N)`) before filling them inside a for loop to avoid costly dynamic resizing.

Example: *to compute the sum of first N integers*

```
N = 10;
s = 0;
for k = 1:N
s = s + k;
end
disp(s);
```

2.2 The `while` loop

This iterative loop is used to repeat a block until a stopping condition is met. It is to be used when the number of iterations is not known in advance.

The general syntax of the `while` loop is as follows:

```
while condition
// body
end
```

When using this iterative loop in Scilab, it is important to note the following:

- We need to always ensure that the loop condition will eventually become false; otherwise the loop may run forever.
- The `while` loop is natural for iterative numerical schemes (e.g. Newton-Raphson, fixed-point, iterative linear solvers), where a convergence check is performed.

Example:

```
tol = 1e-8;
term = 1;
s = term;
k = 1;
while term > tol
term = term / k;   // update for 1/k!
s = s + term;
k = k + 1;
end
disp(s);
```

2.3 Breaking and continuing

In Scilab, the `break` and `continue` statements let a program alter the normal loop flow. A `break` statement causes an immediate exit from the nearest enclosing loop, after which execution resumes at the statement following that loop. A `continue` statement skips the remainder of the current iteration and proceeds to the next one (the loop index is advanced or the loop condition re-evaluated as appropriate). These constructs are handy for early exits (e.g. when a search succeeds) or skipping invalid items, but should be used sparingly, i.e. clear loop conditions and well-structured code are usually preferable.

Example:

```
for k = 1:10
```

```
if k == 5 then
continue;   // skip printing 5
end
if k == 9 then
break;      // stop the loop at 9
end
disp(k);
end
```

The above program is written to display the integers from 1 to 10. The `continue` statement is executed when the integer 5 is reached, when it forces the program to ignore all code below and go into the next iteration, thereby avoiding 5 from being printed. When the integer 9 is reached, the `break` statement forces the program to exit from the loop, thereby displaying integers till 8.

3 Functions in Scilab

Functions are the primary mechanism for packaging behaviour into named, reusable units. They let a program encapsulate a computation, hide implementation details and present a simple interface to callers. Well-designed functions improve readability, make testing easier and are the cornerstone of modular programming and code reuse.

In Scilab a function has its own local workspace so variables inside it do not collide with variables in the caller unless explicitly declared global.

3.1 Function definition

A Scilab function is declared with the `function ... endfunction` block. The outputs are listed on the left-hand side and inputs in parentheses on the right. Functions may return multiple outputs and may be saved in a `.sci` file (one function per file is a common organisation) or placed at the end of a script. Inside the function body all variables are local by default, i.e. these variables exist only for the duration of the function call.

The general syntax of a function definition is as follows:

```
function [out1, out2] = fname(in1, in2)
// body: compute out1 and out2 from in1 and in2
endfunction
```

Example: *to compute the sum and product of two numbers with a single function*

```
function [s, p] = sumprod(a, b)
s = a + b;
p = a * b;
endfunction
```

3.2 Function calling

Calling a function uses its name and passing arguments in order (positional calling). The caller can request one or more outputs; Scilab will return the outputs listed by the function, and unrequested outputs are simply ignored. If a function is defined in a separate `.sci` file, we save that file in the current directory or add its folder to Scilab's path so the interpreter can find it. It is always preferred to have meaningful variable names at the call site so the intent of each argument is clear.

The general syntax of a function call is as follows:

```
[out1, out2] = fname(arg1, arg2);
```

Example: The function defined in the example in §3.1 can be called as follows:

```
[s, p] = sumprod(3, 4);
```

3.3 Passing arguments to functions

Arguments in Scilab are passed by value in the language semantics; for large arrays Scilab uses internal optimisations (copy-on-write) so passing large data is efficient until a modification occurs inside the function. Inputs may be scalars, vectors, matrices or other Scilab objects. Functions should validate input sizes and types and provide helpful error messages.

3.4 Returning values from functions

A function can return multiple values by listing them in square brackets on the left-hand side of the function line. The caller receives those outputs in the same order; if the caller asks for fewer outputs, the remaining outputs are ignored. Document the meaning, shape and units (if any) of each output in the function header so callers know what to expect.

The general syntax for returning value from functions is as follows:

```
function [out1, out2] = fname(in1)
// compute out1 and out2
endfunction

// caller
[a, b] = fname(x);
```

Example:

```
function [rho, theta, phi] = cart2sph(x, y, z)
rho = sqrt(x.^2 + y.^2 + z.^2);
if rho == 0 then
theta = 0; phi = 0; return;
end
theta = acos(z / rho);
phi = atan(y, x);  // two-argument form handles quadrants
endfunction

[r, th, ph] = cart2sph(1, 1, 1);
```

4 Applications

4.1 Series sums

E.g. 1: Write a Scilab function `harmonic_sum(n)` that returns the sum

$$1+\frac{1}{2}+\frac{1}{3}+\cdots+\frac{1}{n}$$

Solution: We have to create a `.sci` file which can be later executed in the console window.

I. Contents of the `harmonic_sum.sci` file:

```
function S = harmonic_sum(n)
    S = 0;
    for i=1:n
        S = S + (1/i)
    end
endfunction
```

II. Execution on the console window:

```
--> exec('harmonic_sum.sci', -1);
--> harmonic_sum(15)
ans =
3.3182290
```

E.g. 2: Write a Scilab function `newton_root(f, df, x0, tol, max_iter)` that implements the Newton-Raphson method to find a root of a function $f(x)$. Provide the function header and the essential loop (concise, runnable code), and briefly state two stopping criteria used in your implementation.

Solution: We have to create a `.sci` file which can be later executed in the console window.

I. Contents of the `newton_root.sci` file:

```
function [root, iter, converged] = newton_root(f, df,
    x0, tol, max_iter)
// Newton-Raphson method to find a root of f(x)
// - f and df: callable (function name or function
   handle) accepted by feval
// - x0: initial guess
// - tol: tolerance for stopping (optional, default 1
   e-8)
// - max_iter: maximum iterations (optional, default
   50)
// Returns:
// - root: computed root (last iterate)
// - iter: number of iterations performed
// - converged: 1 if converged, 0 otherwise
    x = x0;
    converged = 0;
    for iter = 1:max_iter
```

```
        fx = feval(x, f);
        dfx = feval(x, df);
        if abs(dfx) < %eps then
            break;
        end
        dx = fx / dfx;
        x = x - dx;
        if abs(dx) < tol then
            converged = 1;
            break;
        end
        if abs(fx) < tol then
            converged = 1;
            break;
        end
    end
    root = x;
endfunction
```

II. Execution on the console window:

```
          --> exec('newton_root.sci', -1);
          --> deff('y=f1(x)', 'y = x^2 - 2');
          --> deff('y=df1(x)', 'y = 2*x');
          --> [root1, it1, conv1] = newton_root(f1,
  df1, 1.0, 1e-12, 50);
          --> mprintf('Test1: root = %.12f, iter = %d
  , converged = %d\n', root1, it1, conv1);
         Test1: root = 1.414213562373, iter = 6,
  converged = 1
```